

```
import pandas as pd
import random
!pip install faker
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()

random.seed(42)

# -----
# CONFIG
# -----
NUM_REVENUE_RECORDS = 500

CURRENCIES = ["INR", "USD", "SGD"]

REVENUE_TYPES = [
    "Recognized",
    "Deferred",
    "Accrued"
]

REVENUE_STREAMS = [
    "Recurring",
    "One-time"
]

PAYMENT_STATUS = [
    "Paid",
    "Partial",
    "Pending",
    "Overdue"
]

BILLING_FREQUENCY = [
    "Monthly",
    "Quarterly",
    "Annual"
]

COUNTRIES = [
    "India",
    "Singapore",
    "USA",
    "UK",
    "Australia"
]
```

```

# -----
# MASTER REFERENCES
# -----
client_ids = [f"CLT{i:03d}" for i in range(1, 41)]

project_ids = [f"PRJ{i:03d}" for i in range(1, 21)]

contract_ids = [f"CON{i:03d}" for i in range(1, 61)]

# -----
# REVENUE MASTER
# -----
revenue_rows = []

invoice_rows = []

collection_rows = []

forecast_rows = []

profit_rows = []

for i in range(1, NUM_REVENUE_RECORDS + 1):

    revenue_id = f"REV{i:05d}"

    invoice_id = f"INV{i:05d}"

    client_id = random.choice(client_ids)

    project_id = random.choice(project_ids)

    contract_id = random.choice(contract_ids)

    revenue_date = fake.date_between(
        start_date="-1y",
        end_date="today"
    )

    revenue_amount = random.randint(
        50000,
        5000000
    )

    tax_amount = round(
        revenue_amount * 0.18,
        2
    )

```

```

)

net_revenue = revenue_amount - tax_amount

actual_cost = random.randint(
    int(revenue_amount * 0.4),
    int(revenue_amount * 0.8)
)

profit_margin = round(
    ((net_revenue - actual_cost)
     / net_revenue) * 100,
    2
)

outstanding_amount = random.randint(
    0,
    int(revenue_amount * 0.4)
)

collection_days = random.randint(
    5,
    120
)

growth_percent = round(
    random.uniform(-15, 45),
    2
)

forecast_revenue = int(
    revenue_amount * random.uniform(1.05, 1.35)
)

upsell_revenue = random.randint(
    0,
    500000
)

payment_status = random.choice(
    PAYMENT_STATUS
)

currency = random.choice(
    CURRENCIES
)

country = random.choice(

```

```

    COUNTRIES
)

# -----
# REVENUE MASTER
# -----
revenue_rows.append({

    "revenue_id": revenue_id,

    "client_id": client_id,

    "client_name":
        fake.company(),

    "project_id": project_id,

    "project_name":
        fake.bs().title(),

    "invoice_id": invoice_id,

    "contract_id": contract_id,

    "revenue_period":
        revenue_date.strftime("%b-%Y"),

    "revenue_date":
        revenue_date,

    "revenue_amount":
        revenue_amount,

    "currency":
        currency,

    "revenue_type":
        random.choice(REVENUE_TYPES),

    "revenue_stream":
        random.choice(REVENUE_STREAMS),

    "monthly_recurring_revenue_mrr":
        round(revenue_amount / 12, 2),

    "annual_recurring_revenue_arr":
        revenue_amount,

```

```
"gross_revenue":
    revenue_amount,

"net_revenue":
    net_revenue,

"tax_amount":
    tax_amount,

"profit_margin":
    profit_margin,

"recognition_date":
    revenue_date + timedelta(days=5),

"payment_status":
    payment_status,

"outstanding_amount":
    outstanding_amount,

"collection_days":
    collection_days,

"revenue_growth_percentage":
    growth_percent,

"revenue_band":
    (
        "High"
        if revenue_amount > 2000000
        else "Medium"
        if revenue_amount > 500000
        else "Low"
    ),

"forecast_revenue":
    forecast_revenue,

"upsell_revenue":
    upsell_revenue,

"country":
    country,

"billing_frequency":
    random.choice(
        BILLING_FREQUENCY
```

```

    )
})

# -----
# INVOICE MASTER
# -----
invoice_date = revenue_date + timedelta(
    days=random.randint(1, 10)
)

due_date = invoice_date + timedelta(
    days=random.randint(15, 60)
)

invoice_rows.append({

    "invoice_id": invoice_id,

    "client_id": client_id,

    "project_id": project_id,

    "contract_id": contract_id,

    "invoice_date": invoice_date,

    "due_date": due_date,

    "invoice_amount": revenue_amount,

    "tax_amount": tax_amount,

    "discount_amount":
        random.randint(0, 100000),

    "total_payable":
        revenue_amount + tax_amount,

    "invoice_status":
        payment_status,

    "currency":
        currency,

    "invoice_type":
        random.choice([
            "Subscription",
            "Milestone",

```

```

        "One-time",
        "Support"
    ])
})

# -----
# PAYMENT COLLECTION
# -----
paid_amount = (
    revenue_amount
    if payment_status == "Paid"
    else revenue_amount * random.uniform(0.3, 0.8)
)

collection_rows.append({

    "collection_id":
        f"COL{i:05d}",

    "invoice_id":
        invoice_id,

    "client_id":
        client_id,

    "project_id":
        project_id,

    "payment_date":
        due_date + timedelta(
            days=random.randint(-5, 30)
        ),

    "payment_amount":
        round(paid_amount, 2),

    "payment_mode":
        random.choice([
            "Bank Transfer",
            "UPI",
            "Credit Card",
            "Wire Transfer"
        ]),

    "payment_status":
        payment_status,

    "outstanding_amount":

```

```

        outstanding_amount,

        "collection_days":
            collection_days,

        "collector_name":
            fake.name(),

        "remarks":
            fake.sentence()
    })

# -----
# REVENUE FORECAST
# -----
forecast_rows.append({

    "forecast_id":
        f"FRC{i:05d}",

    "client_id":
        client_id,

    "project_id":
        project_id,

    "forecast_period":
        (
            revenue_date + timedelta(days=90)
        ).strftime("%b-%Y"),

    "predicted_revenue":
        forecast_revenue,

    "expected_growth_percentage":
        growth_percent,

    "forecast_confidence_score":
        random.randint(60, 98),

    "forecast_category":
        random.choice([
            "Optimistic",
            "Realistic",
            "Conservative"
        ]),

    "forecast_created_date":

```

```

        revenue_date,

        "forecast_owner":
            fake.name()
    })

# -----
# PROFITABILITY ANALYSIS
# -----
profit_rows.append({

    "analysis_id":
        f"PRF{i:05d}",

    "client_id":
        client_id,

    "project_id":
        project_id,

    "revenue_id":
        revenue_id,

    "gross_revenue":
        revenue_amount,

    "actual_cost":
        actual_cost,

    "net_profit":
        net_revenue - actual_cost,

    "profit_margin_percentage":
        profit_margin,

    "resource_cost":
        random.randint(
            100000,
            1000000
        ),

    "operational_cost":
        random.randint(
            50000,
            500000
        ),

    "tax_cost":

```

```

        tax_amount,

        "discount_cost":
            random.randint(
                0,
                100000
            ),

        "profitability_score":
            random.randint(40, 100),

        "profitability_band":
            (
                "High"
                if profit_margin > 35
                else "Medium"
                if profit_margin > 20
                else "Low"
            ),

        "analysis_date":
            revenue_date + timedelta(days=10)
    })

# -----
# DATAFRAMES
# -----
revenue_df = pd.DataFrame(revenue_rows)

invoice_df = pd.DataFrame(invoice_rows)

collection_df = pd.DataFrame(collection_rows)

forecast_df = pd.DataFrame(forecast_rows)

profit_df = pd.DataFrame(profit_rows)

# -----
# EXPORT CSV
# -----
revenue_df.to_csv(
    "revenue_master.csv",
    index=False
)

invoice_df.to_csv(
    "invoice_master.csv",
    index=False
)

```

```
)  
  
collection_df.to_csv(  
    "payment_collection.csv",  
    index=False  
)  
  
forecast_df.to_csv(  
    "revenue_forecast.csv",  
    index=False  
)  
  
profit_df.to_csv(  
    "profitability_analysis.csv",  
    index=False  
)  
  
print("Revenue CSV files generated successfully!")
```